Visualizador de Backpropagation & Otimizadores

Projeto de Cálculo 3 | Ciência da Computação
Relatório de progresso: o que foi feito e o que falta

1. Visão geral

Ambiente de jogo 2D em JavaScript puro (ES6+), sem bibliotecas de IA. Uma rede neural aprende a jogar por **imitation learning**: o humano grava pares [estado, ação], a rede treina com backpropagation e a interface mostra em tempo real o diagrama da rede, a saída softmax e a curva de Loss, com SGD e Adam intercambiáveis.

Estado atual: os 12 arquivos estão criados, o app roda via `python -m http.server`, os 4 painéis renderizam e o loop de treino funciona. Faltam a validação do aprendizado, o painel da Jacobiana completa e o polimento (seção 5).

2. Estrutura de arquivos (pronta)

Arquivo	Responsabilidade
index.html	Layout com 4 canvas e botões Gravar Ações / Treinar IA / Alternar Otimizador / Resetar
styles/main.css	CSS Grid 2x2 (empilha em telas < 900px)
src/main.js	Integração: fases Especialista/Aprendiz, treino por mini-batch, desenho dos painéis
src/math/matrix.js	dot, transpose, add, hadamard, scale, zeros, toColumn
src/math/activations.js	sigmoid, relu e derivadas, softmax, softmaxJacobian
src/math/nn.js	NeuralNetwork: forward() e backward() (regra da cadeia)
src/math/optimizers.js	SGD e Adam (m, v, correção de viés)
src/game/player.js	Física do jogador, hitbox e getState() normalizado
src/game/engine.js	Loop (requestAnimationFrame, dt real), obstáculos, colisão AABB
src/game/telemetry.js	Gravação [estado, ação] one-hot e amostragem balanceada
src/ui/visualizer.js	Diagrama da rede (brilho proporcional a delta) e painel de matrizes
src/ui/charts.js	Curva de Loss SGD vs Adam, cross-entropy, buffer de 200 pontos

3. O que foi feito, passo a passo

Passo 1: math/matrix.js e activations.js

Núcleo de álgebra linear e ativações. Softmax com subtração do máximo (estabilidade numérica). Jacobiana do softmax $J_{ij} = s_i(\delta_{ij} - s_j)$.

Big-O: dot: O(m·n·p); demais operações: O(m·n); softmaxJacobian: $O(n^2)$

Passo 2: nn.js

Forward guarda z e a de cada camada. Backward: $\delta^{(L)} = y_hat - y$ (atalho softmax + cross-entropy) e $\delta^{(l)} = (W^{(l+1)})^T \delta^{(l+1)}$ hadamard $\sigma'(z^{(l)})$. Teste de mesa mostrou o vanishing gradient da sigmoid ($\delta^1 \sim 0{,}03$ contra $\delta^2 \sim 0{,}48$).

Big-O: forward e backward: O(soma de $n_l \cdot n_{l+1}$)

Passo 2b: optimizers.js

SGD: w = w - lr·g. Adam com momentos m e v e correção de viés; no teste de mesa, na 1ª iteração m_hat recupera o gradiente original e o passo ficou ~ lr.

Big-O: step: O(soma de $n_l \cdot n_{l+1}$) para ambos

Passo 3: player.js

Física de Euler (gravidade, pulo, movimento). Estado normalizado [pos_y, vel_y, dist_obstáculo] para evitar escalas diferentes nos gradientes.

Big-O: update e getState: O(1)

Passo 4: engine.js

Loop com dt medido e limitado a 0,05 s, spawn a cada 1,4 s, colisão AABB, obstáculo mais próximo, getSnapshot().

Big-O: update, colisão e nearest: O(n), n = obstáculos na tela (<= ~5)

Passo 5: telemetry.js

Grava pares [estado, ação one-hot] só durante a fase do especialista. Depois evoluiu para amostragem estratificada por classe (sampleBalancedBatch).

Big-O: record: O(1); sampleBalancedBatch: O(n) + O(batch)

Passo 6: visualizer.js

Layout automático dos neurônios; conexões azuis em repouso e rosas durante o backprop, com brilho proporcional a |δ|. Painel genérico de matriz/vetor (verde positivo, vermelho negativo).

Big-O: diagrama: O(N + E); painel: O(linhas·colunas)

Passo 7: charts.js

Cross-entropy com epsilon contra log(0); buffer deslizante de 200 pontos (custo de desenho constante); SGD em vermelho, Adam em verde.

Big-O: push: O(1) amortizado; draw: O(200)

Passos 8 e 9: index.html, main.css, main.js

Grid 2x2, botões das duas fases, treino de 1 mini-batch (8 amostras) por frame e atualização de todos os painéis no onFrame do engine.

Big-O: treino por frame: O(8·soma de $n_l \cdot n_{l+1}$), o gargalo do sistema

4. Bugs encontrados e corrigidos

Sintoma	Causa	Correção
Números da Jacobiana borrados	Canvas esticado por CSS sem ajustar a resolução interna	fitCanvasToDisplaySize() com devicePixelRatio e clearRect antes de cada desenho
Jogador e obstáculo invisíveis	GROUND_Y fixo em 320, fora da altura real do canvas	groundY calculado a partir da altura do canvas, passado ao Player
Todos os painéis vazios (1ª vez)	engine.js colado com comentários no lugar dos métodos	Arquivo completo reenviado
IA sempre prevendo "none" (saida ~ [0, 0, 0, 0.98])	Dataset desbalanceado: a maioria dos frames não tem tecla pressionada	sampleBalancedBatch (estratificada, com reposição)
Todos os painéis vazios (2ª vez)	const batch declarado duas vezes em trainStep() (SyntaxError em main.js:143)	Removida a declaração antiga com sampleBatch

5. O que ainda falta

Ordenado por prioridade. A ordem segue o que mais afeta a correção e o que a especificação do PDF exige.

Prioridade alta: validar o aprendizado

• **Teste de treino completo:** Resetar, gravar 15 a 20 s usando as 3 ações de propósito, treinar e confirmar que a saída softmax varia e que a Loss tende a cair.

• **Gradiente somado em vez de médio:** em trainStep() os gradientes das 8 amostras são somados, então o passo efetivo vira lr × 8 (0,15 × 8 = 1,2 no SGD). Provavelmente contribui para a curva de Loss muito ruidosa. Correção: dividir por batch.length antes de optimizer.step().

• **Verificação numérica do gradiente (gradient checking):** comparar o gradiente analítico de backward() com (L(w+h) - L(w-h)) / 2h. É o "teste de convergência" do Mês 3 do plano e prova que a regra da cadeia está correta.

• **Reduzir o viés de "none" na gravação:** gravar só quando a ação muda ou 1 a cada N frames (hoje grava todo frame a 60 fps).

Prioridade média: requisitos do PDF ainda não atendidos

• **Painel Jacobiana + Gradiente:** hoje mostra só a saída softmax (4×1). Falta exibir a matriz Jacobiana 4×4 (softmaxJacobian já existe, não está ligada) e o vetor gradiente (ou sua norma por camada), atualizados a cada frame. Incluir rótulos jump / left / right / none.

• **Comparação SGD vs Adam simultânea:** hoje só um otimizador roda por vez, e trocar cria um Adam novo com momentos zerados. Para comparar de fato, rodar duas redes com os mesmos pesos iniciais em paralelo e plotar as duas curvas juntas.

• **Animação do backprop "na direção reversa":** hoje as conexões ficam rosas com brilho fixo. O slide pede piscar da saída para a entrada (varredura por camada, com intensidade proporcional à derivada parcial).

• **Equações vivas no painel:** mostrar com valores numéricos as fórmulas dos slides (w_new = w - lr·gradiente, $\delta^{(l)}$, m_hat e v_hat do Adam).

• **Opção de ativação ReLU** nas camadas ocultas (o slide cita ReLU; a sigmoid sofre vanishing gradient).

Prioridade baixa: qualidade e entrega

• Evitar o forward duplicado em trainStep() (backward() já chama forward internamente).

• Suavizar a curva de Loss com média móvel e ajustar a escala do eixo Y.

• Exportar/importar o dataset em JSON (telemetry.toJSON já existe) para não regravar a cada sessão.

• Indicadores de desempenho da IA: tempo de sobrevivência, acurácia contra o dataset do especialista.

• Testes unitários simples de matrix.js e activations.js (usando os testes de mesa como casos).

• README com instruções de execução, e roteiro de apresentação ligando cada painel ao conceito de Cálculo 3 (Jacobiana, gradiente, regra da cadeia).

• Polímento visual final (Mês 4 do plano): legendas, tooltips, responsividade.

6. Roteiro de teste manual (próximo passo imediato)

#	Ação	O que esperar
1	Servidor: python -m http.server 8000, abrir localhost:8000, Ctrl+Shift+R	Console sem erros; 4 painéis desenhados
2	Resetar, depois Gravar Ações	Badge "Especialista (gravando)"; Dataset sobe a cada frame
3	Jogar 15 a 20 s com Espaço, seta esquerda e seta direita	Dataset em ~900 a 1200 amostras
4	Treinar IA	Teclado deixa de controlar; conexões ficam rosas; Loss começa a ser plotada

#	Ação	O que esperar
5	Observar a saída softmax por 30 a 60 s	Valores variam (não travam em ~[0,0,0,1]); Loss com tendência de queda
6	Alternar para Adam	Curva verde aparece; comparar estabilidade com a vermelha

Cronograma do PDF original: Mês 1 (engine + telemetria), Mês 2 (motor matemático), Mês 3 (backprop, SGD/Adam, testes de convergência), Mês 4 (frontend e polimento). Os módulos dos meses 1 a 3 estão escritos; resta validar a convergência (Mês 3) e completar o painel matemático (Mês 4).